75-Day AI Engineering Roadmap

ML / DL foundations → Transformers → LLMs → RAG → LangGraph → Coding Agents → Production AI → Market-Ready Capstone

Operating model: approximately 7 hours/day (4 at work + 3 at home). The schedule deliberately alternates learning, implementation, project work, retrieval from memory, evaluation and regression sprints.

BUILD & DEPLOY	SOFTWARE ENGINEERING	CODING AGENTS	SHAPE THE BUILD
ML / DL LLMs RAG Agents Evals	Git Testing APIs Data Security Reliability	Specs Planning Context Tools Verification Skills	Problem Users MVP Trade-offs Success metrics

What changed from the earlier 60-day plan? The 75-day version keeps the technical progression but promotes software engineering, evaluation/error analysis, coding-agent practice, spec-driven development, context engineering, product/build shaping, and bounded MCP/Agent Skills work to first-class skills. The goal is capability, not topic count.

The 75-Day Architecture

Days	Phase	Outcome
1–7	Foundations + Sprint	Vectors, matrices, probability, ML workflow; start Jira AI Engineer.
8–14	Classical ML + Evals	Classical models, metrics, error analysis, spec-driven development.
15–21	Deep Learning + PyTorch	Backprop, optimization, PyTorch, CNNs, debugging.
22–28	Transformers + Historical Project	Attention, transformer blocks, LLM internals; Movie Recommender placeholder.
29–35	LLMs + Coding Agents	Tokenization, generation, tools, context engineering, Jira V3.
36–42	RAG + Evaluation	Embeddings, ingestion, vector search, grounded generation, retrieval evals.
43–49	LangGraph + Agents	State, routing, branches, cycles, tools, MCP/Agent Skills.
50–56	Production AI	FastAPI, persistence, Docker, reliability, security, observability, system design.
57–63	Evals + Product + AI-Native Engineering	Regression engineering, verifiers, product shaping, ADRs, agent orchestration.
64–70	Historical Projects + Capstone	Then-vs-now work, real-time inference, Jira final build.
71–75	Final Defense	AR/CV comparison, portfolio, system design, final regression, capstone demo.

The Daily 7-Hour Operating Model

Time	Block	Purpose
~2h	Theory	Books, math, guided lessons, documentation and technical reading.
~2h	Coding	Implement from scratch, experiment, solve exercises, debug.
~1h	Project	Turn the day's concept into a feature of an active project.
~1h	Revision	Spaced repetition, blank-page reconstruction, flashcards and reimplementation.
~1h	Docs / Interview	Official docs, architecture Q&A;, trade-offs and mock interview practice.

The invariant is that passive consumption never dominates the whole day. Implementation-heavy days can shift time toward coding and project work; mathematics-heavy days can shift toward theory and exercises.

Regression Sprints — Learning as Regression Testing

Approximately every seven days, major new material pauses. You retrieve concepts from memory, work from a blank editor/page, debug deliberately broken examples, integrate multiple concepts, answer interview-style questions, and record weak areas. A sprint is passed by evidence, not recognition.

NEW KNOWLEDGE	PRACTICE + PROJECT	REGRESSION SPRINT	FAILURE	RELEARN / REBUILD	CONTINUE
Acquire the concept	Use it in code	Retrieve without notes	Find the exact gap	Repair the gap	Integrate and move forward

Regression test dimensions: knowledge recall, reconstruction, modification/debugging, integration, interview explanation, evaluation evidence, and engineering trade-offs.

Jira → AI Software Engineer: The Longitudinal Project

This project begins early and grows with the curriculum. It is intentionally not a giant agent on Day 5. Each version introduces the next capability only after the underlying engineering concept is learned.

Jira Ticket	Requirement	Spec / Plan	Repository Context	RAG / Tools	Code + Tests	Reviewer	Human Approval
Issue / acceptance criteria	Structured requirement	Spec → plan → validation	Relevant files/docs	Search + tool calls	Implementation + tests	Quality gate + eval	Merge / execute boundary

• **V1:** Jira ticket → structured requirement / code suggestion.

• **V2:** specification → plan → implementation → tests.

• **V3:** repository context + retrieval → relevant code generation.

• **V4:** tools + LangGraph + MCP/skills experiment + reviewer.

• **Final:** requirement analysis → context retrieval → planning → code → tests → evaluation → review → human approval, with bounded permissions and security controls.

Historical Projects — Explicit Placeholders

Movie Recommender	Video Streaming + AR + Hand Gestures
Reserved around Day 24 and revisited Days 64–65. We know only what has been supplied: VAE + recommendation + React. No other historical implementation details are assumed. Later we will compare the old approach with modern embeddings, retrieval, ranking, evaluation and AI application architecture.	Reserved around Days 66 and 71. We know only that it involved video streaming, AR, AR models and hand-gesture control. Later we will compare the historical system with modern CV/DL, real-time inference, latency budgets and deployment.

Books + ChatGPT + Documentation

Use all three, but for different jobs. Books provide durable foundations; ChatGPT provides interactive practice; official documentation is the source of truth for changing APIs and framework behavior.

Resource	Role	Timing
Mathematics for Machine Learning	Math foundations	Days 1–7
3Blue1Brown — Essence of Linear Algebra	Visual intuition	Days 1–4
Hands-On Machine Learning — Géron	Practical ML/DL	Days 6–23
Deep Learning — Goodfellow et al.	Selective deep theory	Days 15–23+
Build a Large Language Model From Scratch — Raschka	Transformer/LLM internals	Days 22–35
AI Engineering — Chip Huyen	LLM application engineering	Days 29–70
Designing Machine Learning Systems — Chip Huyen	Production ML/system design	Days 50–63
Designing Data-Intensive Applications — Kleppmann	Distributed systems reference	Days 52–63 / post-75

Rule: do not try to finish every book. Read selected sections when the day's problem needs them. Use ChatGPT to force retrieval, implementation and debugging rather than replacing the book.

Market-Ready Definition

Capability	Evidence
AI applications	Build and deploy AI systems; measure and steer unpredictable outputs.
Software engineering	Reason about architecture, APIs, data, testing, security, reliability, scalability, cost and latency.
Coding agents	Manage context, specs, planning, tools, verifiers, permissions and agent autonomy.
Shaping the build	Frame problems, define MVPs, acceptance criteria, success metrics and trade-offs.
ML / DL	Explain and implement core ML/DL concepts, train/evaluate models and diagnose failures.
LLMs / Transformers	Explain attention, tokenization, embeddings, decoding and structured generation.
RAG / Agents	Build retrieval/workflows/tools; evaluate retrieval and groundedness; control state and termination.
Production	Expose systems through APIs, persistence, containers, logging, retries, configuration and observability.

Final Capability Loop

PROBLEM	SPEC	CONTEXT	BUILD	EVAL	ERROR ANALYSIS	REGRESSION	SHIP / DEFEND
What should be built?	What exactly must it do?	What information/tools does the system need?	Implement with AI + engineering discipline.	Does it actually work?	Why did it fail?	Did the fix break something else?	Can you deploy it and explain the trade-offs?

The roadmap is deliberately not optimized for maximum topic count. It is optimized for retention, implementation ability, evaluation discipline, engineering judgment and market-facing evidence.

Final capstone: an AI Engineering Assistant centered on Jira: requirement analysis → spec/plan → repository retrieval → tools → code generation → tests → reviewer/evaluation → human approval, with security and observability. Historical project comparisons remain placeholders until the original project material is supplied.

Use the Excel tracker every day. Mark Learn, Build, Explain, Revise, Status and Confidence. On regression days, record the specific weakness and the recovery exercise instead of simply marking the day complete.